



TradePulse Progressive Web App (PWA) – Technical Specification Document

Version: 1.0

Date: September 2026

Platform: Next.js 14+ PWA (Web + Mobile Web)

Status: Draft for Development

1. Executive Summary

TradePulse PWA is a mobile-first progressive web application that provides informal traders, spaza shops, and township-based MSMEs with a lightweight, accessible, and installable business management platform. The PWA will serve as the primary entry point for traders to record sales, manage inventory, track performance, and generate business-readiness profiles—without requiring app store downloads.

Key Advantages of PWA

- **No app store dependency:** Instant access via browser, installable to home screen
- **Lower data usage:** <5 MB vs. 25–80 MB for native apps
- **Faster development:** 4–8 weeks vs. 8–14 weeks for native
- **Cross-platform:** Single codebase for Android, iOS, and desktop
- **Offline capability:** Service workers enable offline functionality
- **Easy updates:** Instant deployment without app store review

Success Metrics

- <3 second first contentful paint (FCP) on 3G networks
- 70%+ installation rate (Add to Home Screen)
- 60%+ repeat usage via home screen icon
- 90%+ offline transaction success rate
- <1% data loss in offline scenarios

2. Target Users & Use Cases

Primary Users

- **Informal traders** accessing via mobile browser (Chrome, Safari, Samsung Internet)
- **Device constraints:** Low-end Android phones (2–4 GB RAM), limited storage
- **Connectivity:** Intermittent 3G/4G, frequent offline periods
- **Data sensitivity:** Users with limited monthly data budgets (500 MB–2 GB)

Key Use Cases

1. **First-time user:** Visits `tradepulse.co.za` → Registers → Adds to home screen
2. **Daily trader:** Opens PWA from home screen → Records voice transaction → Views dashboard
3. **Offline user:** Records multiple transactions offline → Auto-syncs when online
4. **Partner user:** Distributor/bank accesses partner dashboard via desktop browser

3. Functional Requirements

3.1 Core Features (MVP)

FR-PWA-001: Installation & Onboarding

- "Add to Home Screen" prompt (Android: Chrome, iOS: Safari)
- Splash screen with TradePulse logo
- First-time user tutorial (3–5 slides)
- Phone number + OTP registration
- Business profile setup (shop name, location, type)
- PIN setup for quick access

FR-PWA-002: Transaction Recording

- Voice note recording (Web Audio API, up to 60 seconds)
- Text input for transaction description
- Real-time speech-to-text (Web Speech API or server-side)
- AI-powered transaction extraction
- Manual correction and confirmation
- Transaction history with search/filter
- Export transaction list (CSV)

FR-PWA-003: Inventory Management

- Product catalogue (add, edit, delete)
- Automatic stock updates from transactions
- Low-stock alerts (in-app notifications)
- Product categories and tags
- Bulk import via CSV (desktop)
- Stock adjustment workflow

FR-PWA-004: Business Dashboard

- Daily/weekly/monthly sales summaries
- Revenue and profit tracking
- Best-selling products (top 5–10)
- Stock level overview (color-coded)
- Interactive charts (revenue trends, product performance)
- Export reports (PDF/CSV)

FR-PWA-005: Offline Mode

- Full offline transaction recording
- Local storage with IndexedDB
- Background sync queue
- Automatic sync when online
- Offline dashboard (cached data)
- Visual indicator for offline status

FR-PWA-006: Notifications

- Push notifications (Firebase Cloud Messaging)
- Low-stock alerts
- Daily/weekly summary reminders
- Partner messages (opt-in)
- App update notifications
- Notification preferences

FR-PWA-007: Readiness Passport

- Generate business profile (PDF)
- Trading history summary
- Revenue trends and consistency score
- Stock turnover metrics
- Share via WhatsApp, email, download
- Partner access codes (for banks, distributors)

3.2 Advanced Features (Post-MVP)

FR-PWA-008: Multi-language Support

- In-app language switcher
- Voice recognition for isiZulu, isiXhosa, Sesotho
- Translated UI and help content
- Localised product names

FR-PWA-009: Partner Portal

- Distributor dashboard (aggregated data)
- Bank/funder access (with consent)
- ESD programme monitoring
- Government reporting tools
- Role-based access control

FR-PWA-010: Financial Integration

- Payment gateway integration (PayFast, Yoco, Ozow)
- Working capital loan applications
- Invoice generation and sharing
- Expense tracking
- Tax estimation

FR-PWA-011: Desktop Enhancements

- Optimised layout for tablets and desktops
- Keyboard shortcuts for power users
- Advanced reporting and analytics
- Bulk operations (CSV import/export)
- Multi-user support (for larger businesses)

4. Non-Functional Requirements

4.1 Performance

Metric	Target	Measurement
First Contentful Paint (FCP)	<1.8 seconds	Lighthouse, Chrome DevTools
Time to Interactive (TTI)	<3.5 seconds	Lighthouse
Largest Contentful Paint (LCP)	<2.5 seconds	Lighthouse
Cumulative Layout Shift (CLS)	<0.1	Lighthouse
First Input Delay (FID)	<100 ms	Lighthouse
PWA installable	Yes	Lighthouse PWA audit
Offline functionality	Yes	Service worker cache
App size (cached)	<5 MB	Browser dev tools
Time to first transaction	<30 seconds	User timing from load

4.2 Security

Requirement	Implementation
HTTPS	Required (Vercel provides free SSL)
Authentication	OTP + PIN, JWT tokens with secure storage
Data encryption	TLS 1.3 for API, IndexedDB encryption for sensitive data
XSS prevention	Content Security Policy (CSP), React auto-escaping
CSRF protection	SameSite cookies, CSRF tokens
POPIA compliance	Consent management, data minimisation, right to deletion
Secure headers	X-Frame-Options, X-Content-Type-Options, Strict-Transport-Security
Audit logging	All transactions and logins logged

4.3 Reliability

Metric	Target
PWA uptime	99.5% (excluding scheduled maintenance)
Service worker success rate	>99% cache hits
Offline sync success	>99% of queued transactions
Error recovery	Automatic retry with exponential backoff
Graceful degradation	App functional even if service worker fails

4.4 Accessibility

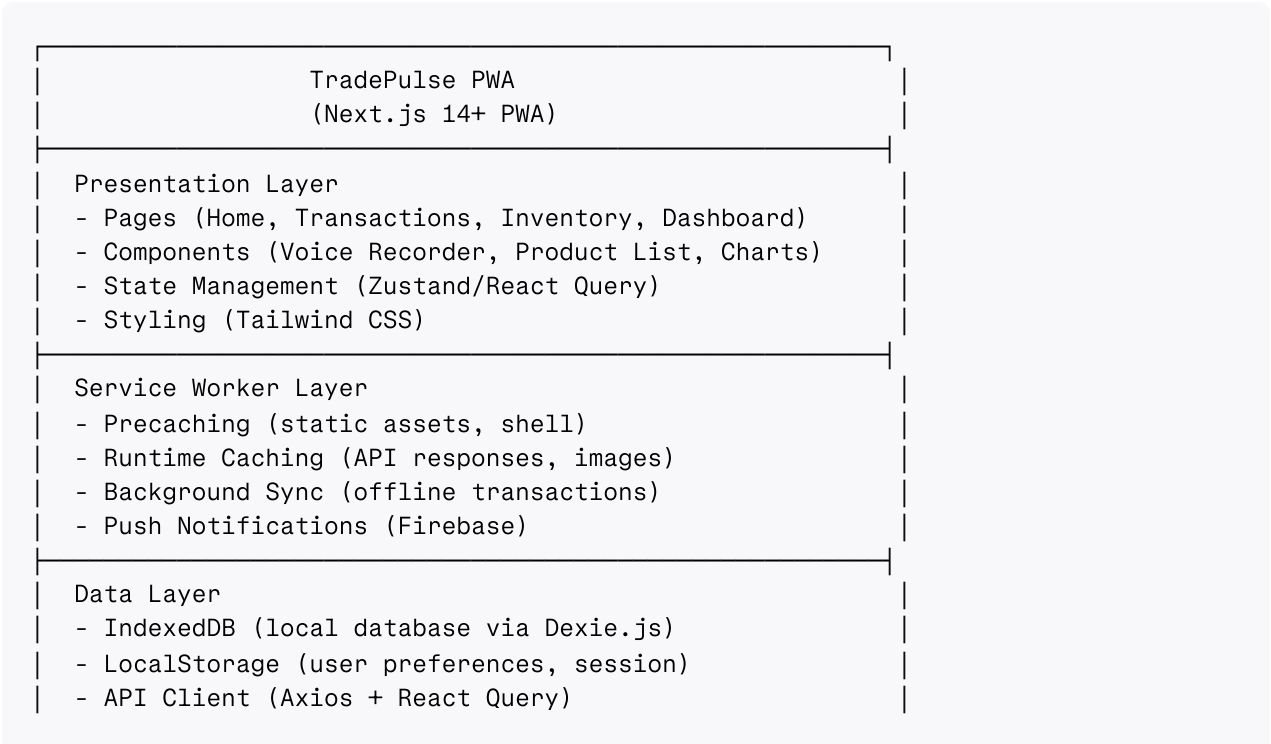
- WCAG 2.1 AA compliance
- Keyboard navigation support
- Screen reader compatibility (NVDA, JAWS, VoiceOver)
- High-contrast mode
- Large text support (up to 200%)
- Focus indicators for interactive elements
- ARIA labels for all interactive elements

4.5 Browser Support

Browser	Minimum Version	Platform
Chrome	88+	Android, Desktop
Safari	15+	iOS, macOS
Samsung Internet	15+	Android
Firefox	90+	Android, Desktop
Edge	88+	Desktop
Opera	75+	Android, Desktop

5. Technical Architecture

5.1 High-Level Architecture



- Cache Manager (images, data, sessions)

HTTPS/TLS

TradePulse Backend
(Node.js + PostgreSQL)
(Shared with Flutter app)

5.2 Next.js Architecture

Framework: Next.js 14+ with App Router

Rendering Strategy:

- Static pages: Marketing, help, about (SSG)
- Dynamic pages: Dashboard, transactions (SSR + client-side hydration)
- API routes: Backend proxy (optional)

State Management:

- **Zustand:** Global app state (user, settings, UI)
- **React Query (TanStack Query):** Server state, caching, sync
- **React Hook Form:** Form handling with validation

Directory Structure

```
tradepulse-pwa/
├── public/
│   ├── icons/
│   ├── images/
│   └── manifest.json
├── src/
│   ├── app/
│   │   ├── layout.tsx
│   │   ├── page.tsx (Home)
│   │   ├── transactions/
│   │   ├── inventory/
│   │   ├── dashboard/
│   │   ├── passport/
│   │   └── api/ (optional proxy routes)
│   ├── components/
│   │   ├── ui/ (buttons, inputs, modals)
│   │   ├── transactions/
│   │   ├── inventory/
│   │   ├── dashboard/
│   │   └── shared/
│   ├── hooks/
│   │   ├── useAuth.ts
│   │   ├── useTransactions.ts
│   │   └── useSync.ts
```

```

|   |   └─ useOffline.ts
|   └─ lib/
|       └─ api.ts (Axios client)
|       └─ db.ts (Dexie.js setup)
|       └─ utils.ts
|       └─ constants.ts
|   └─ stores/
|       └─ userStore.ts
|       └─ uiStore.ts
|       └─ appStore.ts
|   └─ styles/
|       └─ globals.css (Tailwind)
|   └─ types/
|       └─ index.ts (TypeScript types)
|   └─ middleware.ts (Next.js middleware for auth)
└─ service-worker/
    └─ sw.ts (main service worker)
    └─ background-sync.ts
    └─ push-notifications.ts
└─ next.config.js
└─ tailwind.config.js
└─ tsconfig.json
└─ package.json

```

5.3 Service Worker Strategy

Caching Strategy

```

// next.config.js - PWA configuration
const withPWA = require('next-pwa')({
  dest: 'public',
  disable: process.env.NODE_ENV === 'development',
  register: true,
  skipWaiting: true,
  runtimeCaching: [
    {
      urlPattern: /^\/api\/.*\/i,
      handler: 'NetworkFirst',
      options: {
        cacheName: 'api-cache',
        expiration: {
          maxEntries: 100,
          maxAgeSeconds: 60 * 60 * 24, // 24 hours
        },
      },
    },
    {
      urlPattern: /\.?(?:png|jpg|jpeg|svg|gif)$/i,
      handler: 'CacheFirst',
      options: {
        cacheName: 'static-images',
        expiration: {
          maxEntries: 50,
          maxAgeSeconds: 60 * 60 * 24 * 30, // 30 days
        },
      },
    },
  ],
});

```

```

    },
  },
},
],
});

module.exports = withPWA({
  // Next.js config
});

```

Offline Strategy

- **App shell:** Precached (HTML, CSS, JS)
- **API responses:** Network-first with cache fallback
- **Images:** Cache-first with expiration
- **User data:** IndexedDB with background sync

5.4 Data Flow

Online Transaction Flow

```

User opens PWA (home screen or browser)
  ↓
Service worker serves cached app shell
  ↓
User records voice note (Web Audio API)
  ↓
Audio uploaded to backend (S3/R2)
  ↓
Backend transcribes (Whisper API)
  ↓
AI extracts transaction (GPT-4o-mini)
  ↓
Validation layer checks schema + rules
  ↓
Transaction saved to PostgreSQL
  ↓
React Query invalidates cache
  ↓
UI updates automatically

```

Offline Transaction Flow

```

User opens PWA from home screen
  ↓
Service worker detects offline
  ↓
User records voice note
  ↓

```

```
Transaction saved to IndexedDB (Dexie.js)
  ↓
Background sync registered
  ↓
User continues working offline
  ↓
When online: background sync triggers
  ↓
Queued transactions uploaded to backend
  ↓
Backend processes and confirms
  ↓
IndexedDB updated with server IDs
  ↓
React Query cache updated
  ↓
UI reflects synced state
```

6. Technology Stack

6.1 Frontend (PWA)

Component	Technology	Version
Framework	Next.js	14+ (App Router)
Language	TypeScript	5.3+
Styling	Tailwind CSS	3.4+
State Management	Zustand	4.5+
Server State	TanStack Query (React Query)	5.0+
Forms	React Hook Form	7.49+
Validation	Zod	3.22+
Local Database	Dexie.js (IndexedDB wrapper)	4.0+
HTTP Client	Axios	1.6+
PWA Plugin	next-pwa	5.6+
Voice Recording	Web Audio API + MediaRecorder	Native
Speech-to-Text	Web Speech API (optional)	Native
Charts	Recharts or Chart.js	Latest
PDF Generation	@react-pdf/renderer	3.4+
Push Notifications	Firebase Cloud Messaging	Latest
Analytics	Google Analytics 4 + Vercel Analytics	Latest
Error Tracking	Sentry	7.90+

Component	Technology	Version
Icons	Lucide React or Heroicons	Latest
UI Components	shadcn/ui or Radix UI	Latest

6.2 Backend Services

(Shared with Flutter app)

Component	Technology	Provider
Runtime	Node.js	20 LTS
Framework	Express or Fastify	4.x / 4.x
Database	PostgreSQL	15+ (Supabase/Railway)
Cache/Queue	Redis	7+ (Upstash/Redis Cloud)
File Storage	AWS S3 or Cloudflare R2	Standard
AI/ML	OpenAI API (GPT-4o-mini, Whisper)	Latest
Authentication	NextAuth.js or Auth0	Latest
Push Notifications	Firebase Cloud Messaging	Latest
Monitoring	Sentry + Datadog	Latest
Hosting	Vercel (PWA) + Render/Railway (Backend)	N/A

6.3 DevOps & Infrastructure

Component	Technology
Version Control	GitHub
CI/CD	Vercel (automatic deployments)
Hosting	Vercel (PWA), Render/Railway (Backend)
Domain	tradepulse.co.za
SSL	Vercel (free, auto-renewed)
Monitoring	Vercel Analytics, Sentry, Datadog
Logging	Vercel Logs + Datadog
Secrets	Vercel Environment Variables + GitHub Secrets
Backups	Automated daily (PostgreSQL via provider)

7. Data Models

7.1 TypeScript Types

```
// types/index.ts

export interface User {
  id: string;
  phoneNumber: string;
  businessName: string;
  location: string;
  businessType: string;
  createdAt: Date;
  lastActiveAt: Date;
  isVerified: boolean;
  consentFlags: string[];
}

export interface Transaction {
  id: string;
  userId: string;
  type: 'sale' | 'purchase' | 'adjustment';
  productId: string;
  productName: string;
  quantity: number;
  unitPrice: number;
  totalAmount: number;
  timestamp: Date;
  voiceNoteUrl?: string;
  transcription?: string;
  isSynced: boolean;
  serverId?: string;
  localCreatedAt: Date;
}

export interface Product {
  id: string;
  userId: string;
  name: string;
  category: string;
  currentStock: number;
  minStockThreshold: number;
  averagePrice: number;
  lastUpdated: Date;
  isLowStock: boolean;
}

export interface LocalTransaction {
  id: string;
  userId: string;
  type: string;
  productName: string;
  quantity: number;
  unitPrice: number;
  totalAmount: number;
```

```

    timestamp: number;
    voiceNoteBlob?: Blob;
    isSynced: boolean;
    serverId?: string;
    syncedAt?: number;
  }

  export interface SyncQueueItem {
    id: string;
    transactionId: string;
    operation: 'create' | 'update' | 'delete';
    data: Record<string, any>;
    retryCount: number;
    createdAt: number;
    lastAttemptAt?: number;
    error?: string;
  }

```

7.2 IndexedDB Schema (Dexie.js)

```

// lib/db.ts
import Dexie, { Table } from 'dexie';

class TradePulseDB extends Dexie {
  transactions!: Table<LocalTransaction, string>;
  syncQueue!: Table<SyncQueueItem, string>;
  products!: Table<Product, string>;
  settings!: Table<any, string>;

  constructor() {
    super('TradePulseDB');
    this.version(1).stores({
      transactions: 'id, userId, isSynced, localCreatedAt',
      syncQueue: 'id, transactionId, createdAt',
      products: 'id, userId, lastUpdated',
      settings: 'key',
    });
  }
}

export const db = new TradePulseDB();

```

8. API Integration

8.1 API Client (Axios + React Query)

```

// lib/api.ts
import axios from 'axios';
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query';

const api = axios.create({

```

```

    baseURL: process.env.NEXT_PUBLIC_API_URL,
    headers: {
      'Content-Type': 'application/json',
    },
  });

// Request interceptor for auth token
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('accessToken');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

// Response interceptor for error handling
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      // Handle token expiry
      localStorage.removeItem('accessToken');
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);

export default api;

```

8.2 React Query Hooks

```

// hooks/useTransactions.ts
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query';
import api from '@lib/api';

export function useTransactions(userId: string, dateRange?: { from: Date; to: Date }) {
  return useQuery({
    queryKey: ['transactions', userId, dateRange],
    queryFn: async () => {
      const params = new URLSearchParams();
      if (dateRange) {
        params.set('from', dateRange.from.toISOString());
        params.set('to', dateRange.to.toISOString());
      }
      const response = await api.get(`/api/v1/transactions?${params}`);
      return response.data;
    },
    staleTime: 5 * 60 * 1000, // 5 minutes
  });
}

export function useCreateTransaction() {
  const queryClient = useQueryClient();

```

```

return useMutation({
  mutationFn: async (transactionData: any) => {
    const response = await api.post('/api/v1/transactions', transactionData);
    return response.data;
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ['transactions'] });
    queryClient.invalidateQueries({ queryKey: ['dashboard'] });
  },
});
}

```

8.3 Backend Endpoints

(Same as Flutter app - shared backend)

- POST /api/v1/auth/register
- POST /api/v1/auth/verify-otp
- POST /api/v1/transactions
- GET /api/v1/transactions
- POST /api/v1/sync
- GET /api/v1/products
- POST /api/v1/products
- GET /api/v1/dashboard
- POST /api/v1/passport/generate

9. PWA Features Implementation

9.1 Manifest Configuration

```

// public/manifest.json
{
  "name": "TradePulse - Business Management",
  "short_name": "TradePulse",
  "description": "Record sales, manage stock, grow your business",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#1a56db",
  "orientation": "portrait",
  "icons": [
    {
      "src": "/icons/icon-192.png",
      "sizes": "192x192",
      "type": "image/png",
      "purpose": "any maskable"
    }
  ],

```

```

    {
      "src": "/icons/icon-512.png",
      "sizes": "512x512",
      "type": "image/png",
      "purpose": "any maskable"
    }
  ],
  "categories": ["business", "finance", "productivity"],
  "screenshots": [
    {
      "src": "/screenshots/dashboard.png",
      "sizes": "1080x1920",
      "type": "image/png",
      "form_factor": "narrow"
    }
  ],
  "shortcuts": [
    {
      "name": "Record Transaction",
      "short_name": "Record",
      "description": "Quickly record a sale or purchase",
      "url": "/transactions/new",
      "icons": [{ "src": "/icons/record-icon.png", "sizes": "96x96" }]
    },
    {
      "name": "View Dashboard",
      "short_name": "Dashboard",
      "description": "See your business performance",
      "url": "/dashboard",
      "icons": [{ "src": "/icons/dashboard-icon.png", "sizes": "96x96" }]
    }
  ]
}

```

9.2 Service Worker Registration

```

// app/layout.tsx or dedicated PWA component
import { useEffect } from 'react';

export default function PWAProvider({ children }: { children: React.ReactNode }) {
  useEffect(() => {
    if ('serviceWorker' in navigator) {
      window.addEventListener('load', () => {
        navigator.serviceWorker.register('/sw.js', {
          scope: '/',
        }).then((registration) => {
          console.log('SW registered:', registration);
        }).catch((error) => {
          console.log('SW registration failed:', error);
        });
      });
    }
  }, []);
}

```

```

    return <>{children}</>;
  }

```

9.3 Install Prompt

```

// components/InstallPrompt.tsx
import { useState, useEffect } from 'react';

export function InstallPrompt() {
  const [deferredPrompt, setDeferredPrompt] = useState<any>(null);
  const [showPrompt, setShowPrompt] = useState(false);

  useEffect(() => {
    const handleBeforeInstallPrompt = (e: Event) => {
      e.preventDefault();
      setDeferredPrompt(e);
      setShowPrompt(true);
    };

    window.addEventListener('beforeinstallprompt', handleBeforeInstallPrompt);

    return () => {
      window.removeEventListener('beforeinstallprompt', handleBeforeInstallPrompt);
    };
  }, []);

  const handleInstallClick = async () => {
    if (!deferredPrompt) return;

    deferredPrompt.prompt();
    const { outcome } = await deferredPrompt.userChoice;

    if (outcome === 'accepted') {
      setShowPrompt(false);
    }

    setDeferredPrompt(null);
  };

  if (!showPrompt) return null;

  return (
    <div className="fixed bottom-4 left-4 right-4 bg-blue-600 text-white p-4 rounded"
      <p className="font-semibold">Install TradePulse</p>
      <p className="text-sm">Add to your home screen for quick access</p>
      <div className="mt-3 flex gap-2">
        <button
          onClick={handleInstallClick}
          className="bg-white text-blue-600 px-4 py-2 rounded font-semibold"
        >
          Install
        </button>
        <button
          onClick={() => setShowPrompt(false)}
          className="bg-blue-700 text-white px-4 py-2 rounded"

```

```

      >
      Later
    </button>
  </div>
</div>
);
}

```

9.4 Offline Detection

```

// hooks/useOffline.ts
import { useState, useEffect } from 'react';

export function useOffline() {
  const [isOffline, setIsOffline] = useState(false);

  useEffect(() => {
    const handleOnline = () => setIsOffline(false);
    const handleOffline = () => setIsOffline(true);

    setIsOffline(!navigator.onLine);

    window.addEventListener('online', handleOnline);
    window.addEventListener('offline', handleOffline);

    return () => {
      window.removeEventListener('online', handleOnline);
      window.removeEventListener('offline', handleOffline);
    };
  }, []);

  return isOffline;
}

```

9.5 Background Sync

```

// service-worker/background-sync.ts
const SYNC_TAG = 'sync-transactions';

self.addEventListener('sync', (event: any) => {
  if (event.tag === SYNC_TAG) {
    event.waitUntil(syncTransactions());
  }
});

async function syncTransactions() {
  const db = await openDatabase();
  const unsyncedTransactions = await db.transactions
    .filter(t => !t.isSynced)
    .toArray();

  for (const transaction of unsyncedTransactions) {
    try {

```

```

    await fetch('/api/v1/sync', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        transactions: [transaction],
      }),
    });

    await db.transactions.update(transaction.id, {
      isSynced: true,
      syncedAt: Date.now(),
    });
  } catch (error) {
    console.error('Sync failed:', error);
    // Retry logic with exponential backoff
  }
}
}
}

```

10. Performance Optimisation

10.1 Next.js Optimisations

```

// next.config.js
module.exports = {
  images: {
    formats: ['image/avif', 'image/webp'],
    deviceSizes: [640, 750, 828, 1080, 1200],
  },
  experimental: {
    optimizePackageImports: ['lucide-react', 'recharts'],
  },
  compiler: {
    removeConsole: process.env.NODE_ENV === 'production',
  },
};

```

10.2 Code Splitting

- **Route-based splitting:** Next.js automatic code splitting
- **Component lazy loading:** `React.lazy()` for heavy components
- **Dynamic imports:** `import()` for charts, PDF generation
- **Tree shaking:** Remove unused code via TypeScript + bundler

10.3 Image Optimisation

- **Next.js Image component:** Automatic optimisation
- **Formats:** WebP/AVIF for modern browsers
- **Lazy loading:** Below-fold images
- **Responsive images:** Multiple sizes via `srcSet`

10.4 Caching Strategy

```
// lib/cache.ts
import { QueryClient } from '@tanstack/react-query';

export const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 5 * 60 * 1000, // 5 minutes
      gcTime: 10 * 60 * 1000, // 10 minutes
      retry: 2,
      refetchOnWindowFocus: false,
    },
    mutations: {
      retry: 1,
    },
  },
});
```

11. Security Implementation

11.1 Content Security Policy (CSP)

```
// middleware.ts (Next.js)
import { NextResponse } from 'next/server';

export function middleware(request: Request) {
  const nonce = Buffer.from(crypto.randomUUID()).toString('base64');

  const cspHeader = `
    default-src 'self';
    script-src 'self' 'nonce-${nonce}' 'strict-dynamic' https://www.googletagmanager
    style-src 'self' 'unsafe-inline';
    img-src 'self' blob: data: https://images.tradepulse.co.za;
    font-src 'self';
    connect-src 'self' https://api.tradepulse.co.za https://www.google-analytics.com
    frame-src 'none';
    base-uri 'self';
    form-action 'self';
    upgrade-insecure-requests;
  `;
  const response = NextResponse.next();
```

```

response.headers.set('Content-Security-Policy', cspHeader);
response.headers.set('X-Frame-Options', 'DENY');
response.headers.set('X-Content-Type-Options', 'nosniff');
response.headers.set('Referrer-Policy', 'strict-origin-when-cross-origin');
response.headers.set('Permissions-Policy', 'camera=(), microphone=(), geolocation=');

return response;
}

```

11.2 Authentication Flow

```

// hooks/useAuth.ts
export function useAuth() {
  const [user, setUser] = useState<User | null>(null);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
    // Check for existing session
    const token = localStorage.getItem('accessToken');
    if (token) {
      validateToken(token).then((validUser) => {
        setUser(validUser);
        setIsLoading(false);
      });
    } else {
      setIsLoading(false);
    }
  }, []);

  const login = async (phoneNumber: string, otp: string) => {
    const response = await api.post('/api/v1/auth/verify-otp', { phoneNumber, otp })
    const { accessToken, user } = response.data;

    localStorage.setItem('accessToken', accessToken);
    setUser(user);

    return user;
  };

  const logout = () => {
    localStorage.removeItem('accessToken');
    setUser(null);
  };

  return { user, isLoading, login, logout };
}

```

11.3 IndexedDB Encryption

```
// lib/db.ts (with encryption)
import Dexie from 'dexie';
import { encrypt, decrypt } from './crypto';

class TradePulseDB extends Dexie {
  async addEncryptedTransaction(transaction: LocalTransaction) {
    const encrypted = {
      ...transaction,
      sensitiveData: await encrypt(JSON.stringify(transaction)),
    };
    delete (encrypted as any).sensitiveData;

    return this.transactions.add(encrypted);
  }

  async getDecryptedTransaction(id: string): Promise<LocalTransaction> {
    const encrypted = await this.transactions.get(id);
    if (!encrypted?.sensitiveData) return encrypted as LocalTransaction;

    const decrypted = JSON.parse(await decrypt(encrypted.sensitiveData));
    return { ...encrypted, ...decrypted };
  }
}
```

12. Testing Strategy

12.1 Unit Testing

- **Tools:** Jest, React Testing Library
- **Coverage target:** 80%+ of components and utilities
- **Focus areas:**
 - Transaction validation logic
 - Sync conflict resolution
 - Form validation
 - State management

```
npm run test
# Coverage report in /coverage
```

12.2 Integration Testing

- **Tools:** Playwright or Cypress
- **Scenarios:**
 - Full transaction flow (record → transcribe → save → sync)
 - Offline → online transition
 - Authentication flow
 - PWA installation

```
// tests/integration/transaction.spec.ts
import { test, expect } from '@playwright/test';

test('record and sync transaction', async ({ page }) => {
  await page.goto('http://localhost:3000');
  await page.fill('[name="phone"]', '+27123456789');
  await page.click('button[type="submit"]');
  // ... complete flow
});
```

12.3 Lighthouse Audits

```
npm run lighthouse
# Generates performance, PWA, accessibility reports
```

Target scores:

- Performance: 90+
- Accessibility: 95+
- Best Practices: 95+
- SEO: 90+
- PWA: 100 (installable)

12.4 Cross-Browser Testing

Browser	Tool	Coverage
Chrome (Android)	Chrome DevTools + Real device	40%
Safari (iOS)	BrowserStack + Real device	30%
Samsung Internet	Samsung Remote Debugger	15%
Firefox	Firefox DevTools	10%
Desktop Chrome	Chrome DevTools	5%

13. Deployment & CI/CD

13.1 Vercel Deployment

```
// vercel.json
{
  "framework": "nextjs",
  "regions": ["cpt1"], // Cape Town region for SA users
  "env": {
    "NEXT_PUBLIC_API_URL": "https://api.tradepulse.co.za"
  },
  "headers": [
    {
      "source": "/(.*)",
      "headers": [
        {
          "key": "X-Content-Type-Options",
          "value": "nosniff"
        },
        {
          "key": "X-Frame-Options",
          "value": "DENY"
        }
      ]
    }
  ]
}
```

13.2 GitHub Actions Workflow

```
# .github/workflows/deploy.yml
name: Deploy to Vercel

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-node@v3
        with:
          node-version: '20'
      - run: npm ci
      - run: npm run lint
      - run: npm run test -- --coverage
      - run: npm run build

  deploy:
```

```

needs: test
runs-on: ubuntu-latest
if: github.ref == 'refs/heads/main'
steps:
  - uses: actions/checkout@v3
  - uses: amondnet/vercel-action@v20
    with:
      vercel-token: ${ secrets.VERCEL_TOKEN }
      vercel-org-id: ${ secrets.VERCEL_ORG_ID }
      vercel-project-id: ${ secrets.VERCEL_PROJECT_ID }
      vercel-args: '--prod'

```

13.3 Environment Variables

```

# .env.local (development)
NEXT_PUBLIC_API_URL=http://localhost:3001
NEXT_PUBLIC_FIREBASE_API_KEY=...
NEXT_PUBLIC_SENTRY_DSN=...

# Vercel environment (production)
NEXT_PUBLIC_API_URL=https://api.tradepulse.co.za
NEXT_PUBLIC_FIREBASE_API_KEY=...
NEXT_PUBLIC_SENTRY_DSN=...

```

14. Monitoring & Analytics

14.1 Performance Monitoring

- **Vercel Analytics:** Real-time performance metrics
- **Web Vitals:** FCP, LCP, CLS, FID tracking
- **Custom metrics:** Transaction recording time, sync duration

```

// app/layout.tsx
import { Analytics } from '@vercel/analytics/react';
import { SpeedInsights } from '@vercel/speed-insights/next';

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="en">
      <body>
        {children}
        <Analytics />
        <SpeedInsights />
      </body>
    </html>
  );
}

```

14.2 Error Tracking

- **Sentry:** Automatic error capture, source maps
- **Custom error boundaries:** React error boundaries with Sentry integration
- **User context:** Attach user ID, session info to errors

```
// lib/sentry.ts
import * as Sentry from '@sentry/nextjs';

Sentry.init({
  dsn: process.env.NEXT_PUBLIC_SENTRY_DSN,
  environment: process.env.NODE_ENV,
  tracesSampleRate: 0.1,
  replaysSessionSampleRate: 0.1,
  replaysOnErrorSampleRate: 1.0,
  integrations: [
    new Sentry.Replay({
      maskAllText: true,
      blockAllMedia: true,
    }),
  ],
});
```

14.3 User Analytics

- **Google Analytics 4:** Page views, events, conversions
- **Custom events:** Transaction recorded, passport generated, offline usage
- **Privacy-first:** No PII in analytics, respect Do Not Track

```
// lib/analytics.ts
import { GoogleAnalytics } from '@next/third-parties/google';

export function logEvent(eventName: string, eventParams?: Record<string, any>) {
  if (typeof window !== 'undefined' && (window as any).gtag) {
    (window as any).gtag('event', eventName, eventParams);
  }
}
```

15. Maintenance & Support

15.1 Update Strategy

- **Automatic updates:** Service worker updates on app reload
- **Version tracking:** Display app version in settings
- **Force update:** Critical updates trigger mandatory refresh
- **Rollback:** Vercel instant rollback if issues detected

15.2 Browser Compatibility

- **Evergreen browsers:** Auto-update with browser
- **Older browsers:** Graceful degradation (no PWA features, basic functionality)
- **iOS Safari:** Special handling for install prompts, push notifications

15.3 User Support

- **In-app help:** FAQ, tutorial videos, tooltips
- **Contact form:** In-app support request
- **WhatsApp support:** Link to business WhatsApp
- **Status page:** `status.tradepulse.co.za` for uptime monitoring

16. Risk Management

16.1 Technical Risks

Risk	Probability	Impact	Mitigation
Service worker fails	Low	High	Fallback to network-only, error boundaries
IndexedDB quota exceeded	Medium	Medium	Auto-cleanup old data, warn user at 400 MB
iOS Safari limitations	Medium	Medium	Feature detection, graceful degradation
Voice recording fails	Medium	Medium	Fallback to text input, server-side transcription
Slow 3G networks	High	Medium	Aggressive caching, lazy loading, image optimisation

16.2 Business Risks

Risk	Probability	Impact	Mitigation
Low installation rate	Medium	High	In-app prompts, QR codes, distributor training
Users prefer native app	Low	Medium	Highlight PWA benefits (no download, less data)
Browser compatibility issues	Medium	Medium	Extensive testing, polyfills, feature detection
Data costs deter usage	High	High	Offline-first, minimal data usage, compression

17. Timeline & Milestones

Phase 1: MVP (Weeks 1–6)

- Week 1: Project setup, Next.js + PWA configuration
- Week 2: Authentication, user onboarding
- Week 3: Transaction recording (voice + text)
- Week 4: Inventory management, product catalogue
- Week 5: Dashboard, charts, basic reporting
- Week 6: Offline mode, service worker, testing

Phase 2: Polish (Weeks 7–8)

- Week 7: Performance optimisation, Lighthouse audits
- Week 8: Cross-browser testing, bug fixes, beta launch

Phase 3: Scale (Months 3–6)

- Month 3: Partner dashboards, multi-language
- Month 4: Readiness Passport, PDF generation
- Month 5: Push notifications, advanced analytics
- Month 6: Desktop optimisations, financial integrations

18. Budget Estimate

Development Costs

Component	Cost (ZAR)
Next.js PWA development (6 weeks)	R300,000 – R600,000
Backend integration (shared with Flutter)	R200,000 – R400,000
UI/UX design	R100,000 – R200,000
Project management	R100,000 – R200,000
Testing & QA	R100,000 – R200,000
Total Development	R800,000 – R1,600,000

Infrastructure Costs (Monthly)

Component	Cost (ZAR)
Vercel hosting (Pro plan)	R400 – R2,000
Backend hosting (shared)	R2,500 – R10,000
Database (shared)	R1,500 – R5,000

Component	Cost (ZAR)
File storage (S3/R2)	R1,000 – R4,000
AI/ML (OpenAI, Whisper)	R5,000 – R25,000
Monitoring (Sentry, Datadog)	R1,500 – R5,000
Total Monthly	R11,900 – R51,000

19. Success Criteria

Technical Success

- ✔ Lighthouse PWA score: 100
- ✔ Performance score: 90+
- ✔ Accessibility score: 95+
- ✔ First contentful paint: <1.8s on 3G
- ✔ Offline transaction success: >99%

Business Success

- ✔ 500+ PWA installations in 3 months
- ✔ 70%+ home screen retention
- ✔ 60%+ weekly active users
- ✔ <2% uninstall rate
- ✔ R200,000+ monthly recurring revenue in 6 months

20. Comparison: PWA vs. Flutter App

Factor	PWA	Flutter App
Development cost	R800k–R1.6M	R2M–R3.5M
Timeline	6–8 weeks	12–16 weeks
App size	<5 MB	25–50 MB
Updates	Instant (no app store)	App store review required
Offline support	Limited (IndexedDB)	Full (native database)
Performance	85–95% of native	Near-native
Voice recording	Web Audio API (good)	Native APIs (better)
Push notifications	FCM (Android good, iOS limited)	FCM (full support)
Biometric auth	WebAuthn (limited)	Native biometrics (full)

Factor	PWA	Flutter App
App store presence	No	Yes (Play Store, App Store)
Data usage	Lower (web-optimised)	Higher (app downloads)
Maintenance	Lower (single codebase)	Higher (iOS + Android)

Recommendation: Launch with PWA first (faster, cheaper, lower risk), then build Flutter app once you have traction and funding.

21. Approval & Sign-off

Role	Name	Signature	Date
Product Owner			
Technical Lead			
Project Manager			
Security Officer			
Stakeholder			

Document Control:

- **Version:** 1.0
- **Status:** Draft
- **Next Review:** October 2026
- **Distribution:** Development Team, Stakeholders, Investors

This PWA technical specification complements the Flutter app specification, providing a comprehensive blueprint for building a lightweight, accessible, and high-performance progressive web application. The PWA approach reduces development costs and time-to-market while maintaining most of the functionality of a native app—making it ideal for the South African township market where data costs and device storage are key constraints.